SOFTWARE ENGINEERING TESTING DELIVERABLE

Electronic Store Inventory Management System

Unit Test Report

Project Title	Electronic Store Inventory Management System	**Document Title**	Unit Test Report
Testing Date	November 6, 2026 (2026-11-06)	**Test Runner**	Pytest 9.1.1 (Python 3.14.6)
Architecture	Three-Tier Layered Architecture (Presentation, Business, Data)	**Application Stack**	Flask 3.1.3, SQLAlchemy 2.1.6, SQLite, bcrypt 5.0.0
Test Command	`./venv/bin/python -m pytest -v`	**Suite Outcome**	**323 Passed / 0 Failed / 0 Errors / 0 Skipped (105.34s)**

1. Introduction

This Unit Test Report documents the automated verification of the **Electronic Store Inventory Management System**. The application is structured as a three-tier layered web application comprising a **Presentation Layer** (`app/presentation/` Flask blueprints, route views, and role-authorization decorators), a **Business Logic Layer** (`app/business/` services for sales processing, stock adjustments, product returns, purchase order lifecycle, password hashing, audit logging, and report exports), and a **Data Access Layer** (`app/data/` SQLAlchemy ORM models and database extensions), alongside operational maintenance scripts (`scripts/backup_database.py`).

Scope Clarification (Unit vs. Multi-Layer Automated Tests): While this deliverable is titled *Unit Test Report* in accordance with the course submission specification, an inspection of the actual test suite (`tests/`, 323 automated tests across 11 modules) establishes that the suite is a comprehensive automated verification suite rather than solely isolated mock-only unit tests. Specifically, the 323 executed tests encompass three distinct verification levels:

- **Isolated Unit Tests:** Direct function and model-property tests that verify pure computation and validation logic without HTTP requests, such as bcrypt password hashing and 72-byte boundary validation (`app/business/security.py`), low-stock boundary properties and suggested reorder quantities (`Product.is_below_reorder_threshold`, `Product.suggested_reorder_quantity`), and `Supplier` model serialization.
- **Service-to-Database Integration Tests:** Business-service tests that invoke domain functions directly (`create_sale()`, `restock_product()`, `restock_inventory()`, `process_return()`, `create_purchase_order()`, `receive_purchase_order()`, `record_partial_delivery()`, and `create_purchase_order_from_low_stock()`) against an in-memory SQLite database to verify atomic commits, multi-record updates, and transaction rollbacks on mid-operation failures.
- **HTTP Route, Session, and Role-Based Access Control (RBAC) Tests:** Presentation-to-database tests using Flask's `test_client()` that exercise request parsing, session authentication, `@admin_required` and `@manager_required` access guards, HTML template rendering, and database persistence across all application routes.

2. Testing Objectives

The primary objectives of executing the automated test suite are to:

- **Verify Functional Correctness:** Confirm that all core business operations—product CRUD, SKU uniqueness, category and stock-status filtering, multi-item sales checkout, 5% tax and invoice calculation, returns/refunds, supplier management, purchase order creation/receipt/partial delivery, and date-range reporting—produce accurate outputs.

- **Validate Input Rules and Boundary Conditions:** Ensure that blank strings, negative prices or stock counts, non-integer quantities, duplicate SKUs or return IDs, reversed report date ranges, and over-delivery or overselling attempts are deterministically rejected with clear validation errors.
- **Confirm Transactional Atomicity and Data Integrity:** Verify that multi-item sales, batch restocks, and purchase order receipts commit atomically and roll back completely if a validation error or simulated database failure occurs mid-operation.
- **Enforce Authentication and Role-Based Access Control (RBAC):** Verify that passwords are stored exclusively as salted bcrypt hashes, deactivated accounts cannot sign in, and server-side role checks (`staff`, `manager`, `admin`) enforce HTTP 302/403 restrictions on protected routes.
- **Verify Audit Trail and Backup Operations:** Confirm that product stock edits, price changes, and deletions record actor `user_id` and timestamps in `AuditLog`, and that `scripts/backup_database.py` produces verified SQLite backups passing `PRAGMA integrity_check`.

3. Testing Environment

The automated test suite was executed in the repository's local Python virtual environment. Table 1 records the exact hardware, operating system, interpreter, and package versions used during execution.

Environment Component	Specification / Version
Operating System	macOS 27.2 (Build 26B123) — Darwin Kernel Version 27.2.0 (`arm64`)
Python Interpreter	Python 3.14.6 (`/Users/udayk/Desktop/Electronics-store-inventory-management-system/venv/bin/python`)
Virtual Environment	Repository local venv (`./venv`)
Test Framework	`pytest 9.1.1` (with `pluggy 1.7.0`, `iniconfig 2.3.1`)
Web & Auth Framework	`Flask 3.1.3`, `Werkzeug 3.1.9`, `Flask-Login 0.6.3`, `Flask-WTF 1.3.0`, `WTForms 3.3.1`, `Jinja2 3.1.6`, `itsdangerous 2.2.0`
ORM & Database Engine	`Flask-SQLAlchemy 3.1.1`, `SQLAlchemy 2.1.6`, `PyMySQL 1.2.3`, SQLite 3 (in-memory `sqlite:///:memory:` & `tmp_path` file DB)
Security & Utilities	`bcrypt 5.0.0`, `python-dotenv 1.2.4`, `reportlab 5.1.0`

4. Testing Methodology

1. Test Discovery and Execution: Pytest was invoked from the repository root using the command `./venv/bin/python -m pytest -v`. Pytest automatically discovered all 11 `test_*.py` modules inside `tests/` and collected 323 test items (including expanded `@pytest.mark.parametrize` cases).

2. Fixtures and Test Isolation (`tests/conftest.py`): Every test executes against a clean, isolated database state provided by shared fixtures in `tests/conftest.py`: (a) `app` creates a Flask application instance configured with `TESTING=True`, `SECRET_KEY='test-secret'`, and `SQLALCHEMY_DATABASE_URI='sqlite:///:memory:'`, runs `db.create_all()` prior to yielding, and executes `db.session.remove()` and `db.drop_all()` on teardown; (b) `client` returns `app.test_client()` for simulating HTTP GET/POST/DELETE requests; and (c) `logged_in_admin` (`test-admin`), `logged_in_manager` (`test-manager`), and `logged_in_staff` (`test-staff`) seed role-specific accounts with bcrypt-hashed passwords and authenticate the test client session via `POST /login`.

3. Fault Injection, Mocking, and Parametrization: To test error recovery and external boundaries without side effects, the suite uses Pytest's `monkeypatch` fixture to simulate database commit failures (`db.session.commit` raising exceptions during product deletion, purchase order receipt, partial delivery, and low-stock PO creation) and schema creation failures (`Supplier.__table__.create`). `unittest.mock.patch` is used in `tests/test_auth.py` to intercept outbound SMTP calls in `send_reset_email()`. `@pytest.mark.parametrize` is used across `tests/test_sales.py` and `tests/test_purchase_orders.py` to test invalid quantity inputs (0, negative numbers, decimals, strings, booleans, None).

4. Assertion Strategy: Tests verify both presentation responses and underlying database state: assertions check HTTP status codes (`200`, `201`, `302`, `400`, `403`, `404`, `405`, `409`), flashed feedback banners, JSON payloads, and direct SQLAlchemy queries confirming exact `Decimal` currency totals, stock levels, `AuditLog` records, and zero partial updates after rolled-back transactions.

5. Modules Tested

Table 2 lists all 11 test modules in the `tests/` directory alongside the application modules and functional capabilities verified by each module.

Test Module	Functionality Tested
`tests/test_app.py`	Application factory (`create_app()`) initialization, dashboard home route (`GET /`) HTTP 200 response, and startup error handling when supporting table creation fails.
`tests/test_auth.py`	User authentication (`GET/POST /login`, `POST /logout`), valid/invalid credentials, deactivated account blocking, Admin user management (`/users`, `/users/create`, `/users/<id>/edit`, deactivate), username/password/role validation, Staff RBAC restrictions, and token-based password reset (`/forgot-password`, `/reset-password/<token>`).
`tests/test_backup_database.py`	SQLite database backup script (`scripts/backup_database.py:create_backup()`), verifying backup file creation in `BACKUP_DIR`, preservation of source table records, `PRAGMA integrity_check == ('ok',)`, and `FileNotFoundError` handling when the source database file is missing.
`tests/test_products.py`	Product CRUD (`/products/create`, `/products/<id>/edit`, `/products/<id>/update`, `/products/<id>/delete`), SKU uniqueness validation, sales-history deletion protection, search by name/SKU (`?q=`), filtering by category and stock status (`in_stock`, `low_stock`, `out_of_stock`), reorder threshold configuration (`/products/<id>/reorder-threshold`) and RBAC, product-to-supplier linking, dashboard low-stock alert banners, and `AuditLog` creation, anti-spoofing, and 6-month retention.
`tests/test_purchase_orders.py`	Purchase order business logic (`app/business/purchase_order.py`) and HTTP/JSON routes (`/purchase-orders/*`): PO creation with `Pending` status without premature stock changes, duplicate item aggregation, full PO receipt (`receive_purchase_order()`) atomically increasing stock and recording `ProductRestock`, partial deliveries (`record_partial_delivery()`) transitioning status to `Partially Received` or `Received`, over-delivery prevention, direct PO creation from active low-stock alerts (`create_purchase_order_from_low_stock()`), duplicate pending PO prevention, rollback on simulated DB failures, and Manager/Admin RBAC enforcement.

Test Module	Functionality Tested
`tests/test_revenue_report.py`	Revenue report (`GET /reports/revenue`) access control (Manager/Admin allowed, Staff HTTP 403), date range validation (missing, invalid, or reversed dates), end-of-day boundary inclusion (`23:59:59`) and next-day exclusion, exact `Decimal` revenue summation, daily revenue breakdown, empty-period state, and refund limitation notice.
`tests/test_sales.py`	Sales and inventory stock service functions (`create_sale()`, `process_return()`, `restock_product()`, `restock_inventory()`) and routes (`POST /sales`, `GET /sales/<id>/invoice`, `POST /products/<id>/restock`, `POST /returns`): 5% tax and subtotal/grand-total calculation, automatic stock deduction, overselling prevention, multi-item atomic rollback, invoice rendering, sale-linked and standalone returns, duplicate `return_id` blocking, original-sale-price refund calculation, and batch restock atomicity.
`tests/test_sales_report.py`	Sales report (`GET /reports/sales`) RBAC enforcement (Manager/Admin HTTP 200, Staff HTTP 403), start/end date validation, date boundary filtering, empty-state message, and rendering of matching sales transactions, totals, and invoice links.
`tests/test_security.py`	Unit tests for `hash_password()` and `verify_password()` in `app/business/security.py`: verifying non-plaintext storage (`$2` bcrypt prefix), correct/incorrect password verification, unique per-hash salt generation, malformed hash rejection, and 72-byte UTF-8 bcrypt limit enforcement.
`tests/test_stock_report.py`	Stock movement report (`GET /reports/stock`) RBAC enforcement (Manager/Admin HTTP 200, Staff HTTP 403), date range validation, and per-product aggregation of `Units Sold`, `Units Restocked`, `Units Returned`, `Net Movement` (`restocked + returned - sold`), and `Current Stock`.
`tests/test_suppliers.py`	Supplier model unit tests (`contact_info` / `contact_information` compatibility, `to_dict()`, `__repr__()`), Admin-only RBAC on `/suppliers*` routes (blocking anonymous, Staff, and Manager users), supplier creation and editing, field length validation (name ≤ 200, contact ≤ 255, address ≤ 500), alphabetical listing, and 404 handling.

6. Test Execution Summary

The complete automated test suite was executed on **November 6, 2026** using the command `./venv/bin/python -m pytest -v`. Table 3 summarizes the overall execution metrics captured directly from the Pytest terminal session.

Metric	Actual Result
Total Tests	**323**
Passed	**323**
Failed	0
Errors	0
Skipped	0
Execution Time	**105.34s (0:01:45)**

7. Module-wise Results

Table 4 presents the exact breakdown of executed, passed, failed, error, and skipped tests for each of the 11 test modules in `tests/`.

Test Module	Tests Executed	Passed	Failed	Errors	Skipped
`tests/test_app.py`	2	**2**	0	0	0
`tests/test_auth.py`	22	**22**	0	0	0
`tests/test_backup_database.py`	2	**2**	0	0	0
`tests/test_products.py`	99	**99**	0	0	0
`tests/test_purchase_orders.py`	80	**80**	0	0	0
`tests/test_revenue_report.py`	12	**12**	0	0	0
`tests/test_sales.py`	53	**53**	0	0	0
`tests/test_sales_report.py`	10	**10**	0	0	0
`tests/test_security.py`	6	**6**	0	0	0
`tests/test_stock_report.py`	8	**8**	0	0	0
`tests/test_suppliers.py`	29	**29**	0	0	0
Total	**323**	**323**	**0**	**0**	**0**

8. Failed Test Details

No test failures or execution errors occurred. All **323** collected tests across the 11 test modules passed (`323 passed in 105.34s`, with 0 failed, 0 errors, and 0 skipped).

9. Conclusion

The automated Pytest execution confirms that all 323 implemented tests across the 11 test modules in `tests/` pass without failures or errors. Specifically, the results establish:

- **Core Business Logic & Model Integrity:** Password hashing (`bcrypt`), sales total and 5% tax calculations, stock deduction and replenishment, sale-linked and standalone product returns, reorder threshold boundary checks, and purchase order lifecycle operations (`Pending`, `Partially Received`, `Received`) behave as specified.
- **Transactional Atomicity & Validation:** Invalid inputs, duplicate SKUs, duplicate return IDs, overselling requests, and over-deliveries are deterministically rejected, and simulated mid-transaction database failures roll back cleanly without partial state corruption.
- **Route & RBAC Enforcement:** Flask routes across `auth_bp`, `main_bp`, `supplier_bp`, and `purchase_order_bp` properly enforce `staff`, `manager`, and `admin` role restrictions, audit logging, and date-range reporting.
- **Scope Boundaries:** Because the automated Pytest suite executes against Flask's in-process `test_client()` and temporary SQLite databases (`sqlite:///:memory:` and `tmp_path`), system-level non-functional concerns such as live multi-user concurrency load testing, production HTTPS/TLS deployment, and live SMTP delivery are outside the scope of this unit and integration test suite.